Rapport de TP Clusterisation de conteneurs Talha Tecim

Github : https://github.com/plvo/esgi-2603-my-favorite-places

Images sur Docker Hub : `plvo/mfp-server:1.0` et `plvo/mfp-client:1.0`

État final du cluster

```
$ docker exec swarm-manager-1 docker service ls --format "table
{{.Name}}\t{{.Mode}}\t{{.Replicas}}\t{{.Image}}"
NAME                 MODE         REPLICAS   IMAGE
authelia_authelia    replicated   1/1        authelia/authelia:4.39
dozzle_dozzle        global       4/4        amir20/dozzle:v11.4.0
mfp_client           replicated   2/2        plvo/mfp-client:1.0
mfp_db               replicated   1/1        postgres:17
mfp_server           replicated   2/2        plvo/mfp-server:1.0
portainer_agent      global       4/4        portainer/agent:lts
portainer_portainer  replicated   1/1        portainer/portainer-ce:lts
traefik_traefik      replicated   1/1        traefik:v3.6
traefik_whoami       replicated   1/1        traefik/whoami:latest
vote_db              replicated   1/1        postgres:15-alpine
vote_redis           replicated   1/1        redis:alpine
vote_result          replicated   1/1        dockersamples/examplevotingapp_result:latest
vote_vote            replicated   2/2        dockersamples/examplevotingapp_vote:latest
vote_worker          replicated   2/2        dockersamples/examplevotingapp_worker:latest
```

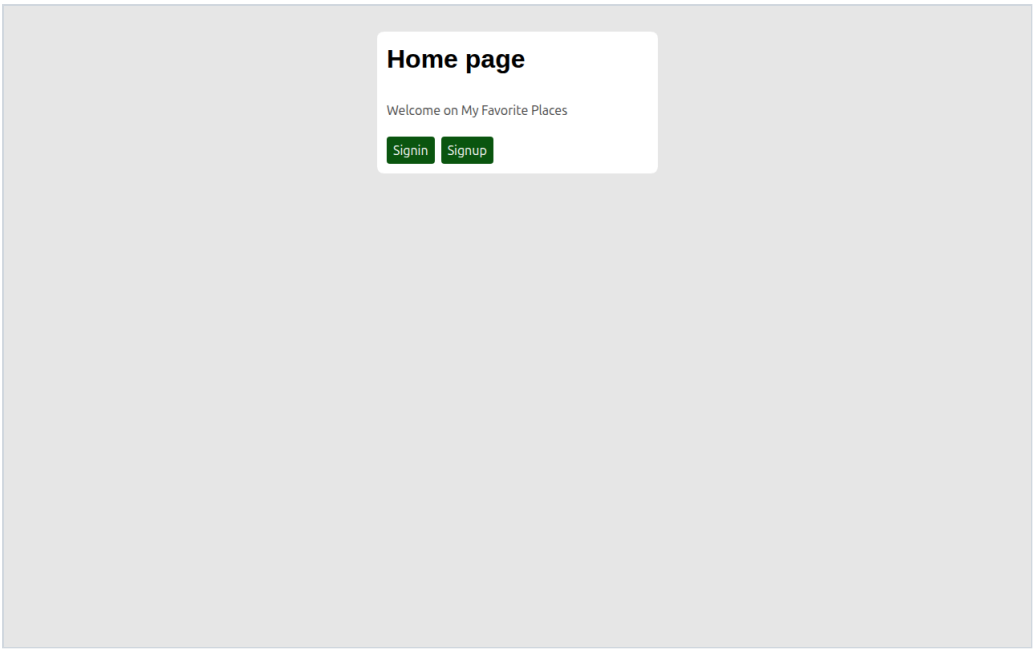
Cours 1 Dockerisation

TD 1

J'ai écrit `server/Dockerfile`, `client/Dockerfile` et un `compose.yml` qui lance le serveur, le client et PostgreSQL.

```
$ docker compose ps
SERVICE   STATUS                   PORTS
client    Up 6 seconds             0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
db        Up 8 seconds (healthy)   5432/tcp
server    Up 6 seconds             0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp

$ curl -X POST http://localhost:8080/api/users -H 'Content-Type: application/json' -d
'{"email":"demo@local.test","password":"demo-password"}'
{"item":{"id":1,"email":"demo@local.test","hashedPassword":"<hash>","createdAt":"2026-11-
04T11:50:40.933Z"}}
HTTP 200
```



La BDD et le serveur communiquent en TCP sur le réseau créé par compose. Le serveur joint la BDD avec le nom du service `db` (DNS interne de Docker) sur le port 5432, qui n'est pas publié sur l'hôte. Avec `localhost` le serveur se chercherait lui-même.

Pour le front, le navigateur appelle `/api/...` sur le port du front et nginx relaie vers `http://server:3000`, donc pas de CORS à gérer.

TD 2

Pour que le serveur attende la BDD j'ai mis un healthcheck `pg_isready` sur `db` et `depends_on` avec `condition: service_healthy` sur `server`.

Les trois services ont `restart: unless-stopped`.

```
$ docker inspect <conteneur> --format '{{.HostConfig.RestartPolicy.Name}}'
db unless-stopped
server unless-stopped
client unless-stopped
```

Pour les build stages, le serveur est compilé dans un stage `build` et l'image finale garde seulement `dist/` et les dépendances de prod, sur une base alpine.

Image serveur	Taille
`node:22-slim` multi-stage	550 Mo
`node:22-alpine` multi-stage, cache yarn vidé	334 Mo

L'image client fait 94 Mo, il ne reste que nginx et les fichiers statiques.

Avant le cours 2, push des images

`docker stack deploy` ne build pas les images et chaque noeud doit pouvoir les pull, donc je les ai mises sur Docker Hub.

```
docker build -t plvo/mfp-server:1.0 server
docker build -t plvo/mfp-client:1.0 client
docker push plvo/mfp-server:1.0
docker push plvo/mfp-client:1.0
```

J'ai corrigé deux choses à ce moment. Sans BDD le serveur s'arrêtait avec le code 0, donc une restart policy `on-failure` dans Swarm ne l'aurait jamais relancé, il sort maintenant avec le code 1. Et j'ai enlevé le `COPY . .` du Dockerfile client comme conseillé dans le cours.

Cours 2 Docker Swarm

TD 1 DinD / DooD

DinD (Docker in Docker) fait tourner un daemon Docker complet dans le conteneur, les conteneurs qu'on lance sont dans ce daemon. Il faut `--privileged`, ce qui casse l'isolation avec l'hôte.

DooD (Docker out of Docker) monte le socket `/var/run/docker.sock` de l'hôte dans le conteneur. Les conteneurs lancés tournent sur le daemon de l'hôte, à côté du conteneur. C'est ce que font Traefik et Portainer, et beaucoup de CI.

Pour tester Swarm il faut DinD, chaque conteneur a son propre daemon donc il se comporte comme une machine à part. Avec DooD il n'y aurait qu'un seul daemon donc un seul noeud. On lance les "machines" avec compose sur un même réseau, à garder en local à cause de `--privileged`.

TD 2 Création du cluster

1. `swarm/compose.yml`

```
services:
  manager:
    image: docker:dind
    privileged: true
    hostname: manager
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /home/manager

  node:
    image: docker:dind
    privileged: true
    scale: 3
```

Le port 443 a été ajouté plus tard pour le cours 5. Le volume anonyme sur `/home/manager` garde les fichiers copiés dedans.

```
$ docker compose up -d
$ docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}" --filter name=swarm-
NAMES             IMAGE         STATUS                  PORTS
swarm-manager-1   docker:dind   Up Less than a second   0.0.0.0:80->80/tcp, [::]:80->80/tcp, 2375-
2376/tcp
swarm-node-1      docker:dind   Up Less than a second   2375-2376/tcp
swarm-node-3      docker:dind   Up Less than a second   2375-2376/tcp
```

```
swarm-node-2      docker:dind   Up Less than a second   2375-2376/tcp

$ docker exec swarm-node-1 docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

`docker ps` marche dans le conteneur et ne liste rien, c'est bien un autre daemon que celui de mon PC.

2. Init du swarm sur le manager

```
$ docker exec swarm-manager-1 docker swarm init
Swarm initialized: current node (rvkpoi5q67x28p1343k9mgi6s) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token <TOKEN> 172.21.0.2:2377
```

3. Les workers rejoignent avec le nom du service `manager` et pas l'IP

```
$ docker exec swarm-node-1 docker swarm join --token <TOKEN> manager:2377
This node joined a swarm as a worker.
$ docker exec swarm-node-2 docker swarm join --token <TOKEN> manager:2377
This node joined a swarm as a worker.
$ docker exec swarm-node-3 docker swarm join --token <TOKEN> manager:2377
This node joined a swarm as a worker.
```

4. Liste des noeuds

```
$ docker exec swarm-manager-1 docker node ls
ID                            HOSTNAME       STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
e28xwqci5wxflocjog7066yj9     d0ada8f4a045   Ready     Active                          29.9.3
59n2ucz29gt124ptp8ojujrb6     ddc4456364e2   Ready     Active                          29.9.3
omgnnm2hagod7rs8effm3ze9d     de69559d8a5a   Ready     Active                          29.9.3
rvkpoi5q67x28p1343k9mgi6s *   manager        Ready     Active         Leader           29.9.3
```

TD 3 Tests du cluster

1 et 2. `swarm/hello-world.compose.yml`, copié avec `docker cp` dans `/home/manager`

```
services:
  hello-world:
    image: hello-world
    deploy:
      replicas: 2
```

3.

```
$ docker exec swarm-manager-1 docker service ls
ID             NAME                MODE         REPLICAS   IMAGE                PORTS
4pv9hm20b6r6   hello_hello-world   replicated   0/2        hello-world:latest

$ docker exec swarm-manager-1 docker service ps hello_hello-world ...
NAME                      NODE           DESIRED STATE   CURRENT STATE                      ERROR
hello_hello-world.1       de69559d8a5a   Ready           Preparing less than a second ago
 \_ hello_hello-world.1   de69559d8a5a   Shutdown        Complete less than a second ago
 \_ hello_hello-world.1   de69559d8a5a   Shutdown        Complete 6 seconds ago
 \_ hello_hello-world.1   de69559d8a5a   Shutdown        Complete 11 seconds ago
 \_ hello_hello-world.1   manager        Shutdown        Complete 17 seconds ago
 \_ hello_hello-world.1   manager        Shutdown        Complete 22 seconds ago
```

Avec `docker ps -a` sur chaque noeud on voit des conteneurs `Exited (0)` sur le manager, node-1 et node-2, et rien sur node-3.

Swarm répartit les tâches tout seul sur les noeuds, pas forcément sur tous. hello-world affiche son message puis s'arrête, et comme Swarm veut garder 2 replicas il le relance en boucle, parfois sur un autre noeud. Du coup le service reste à 0/2. Un service répliqué doit être un process qui tourne en continu.

4. Pour les variantes j'ai pris `nmatsui/hello-world-api` (l'image utilisée dans le repo du prof) parce qu'elle reste allumée.

- `hello-replicas.compose.yml` avec 2 replicas, 2/2 sur manager et de69559d8a5a
- `hello-manager.compose.yml` avec `node.role == manager`, 1/1 sur manager
- `hello-global.compose.yml` avec `mode: global`, 4/4 avec une tâche par noeud

```
$ docker exec swarm-manager-1 docker service ps helloglobal_hello --filter desired-state=running ...
NAME                                      NODE           CURRENT STATE            ERROR
helloglobal_hello.59n2ucz29gt124ptp8ojujrb6   ddc4456364e2   Running 20 seconds ago
helloglobal_hello.e28xwqci5wxflocjog7066yj9   d0ada8f4a045   Running 20 seconds ago
helloglobal_hello.omgnnm2hagod7rs8effm3ze9d   de69559d8a5a   Running 24 seconds ago
helloglobal_hello.rvkpoi5q67x28p1343k9mgi6s   manager        Running 24 seconds ago
```

Cours 4 Traefik et Portainer

J'ai ajouté cette ligne dans `/etc/hosts`, et j'ai dû arrêter apache qui prenait déjà le port 80 sur mon PC.

```
127.0.0.1 traefik.swarm.localhost whoami.swarm.localhost vote.swarm.localhost result.swarm.localhost
portainer.swarm.localhost mfp.swarm.localhost api.swarm.localhost dozzle.swarm.localhost
authelia.swarm.localhost
```

Exercice 1 Traefik

`swarm/traefik.yml` c'est la stack donnée dans le cours. Traefik lit le socket Docker donc il tourne sur le manager, c'est lui qui connaît l'état du swarm.

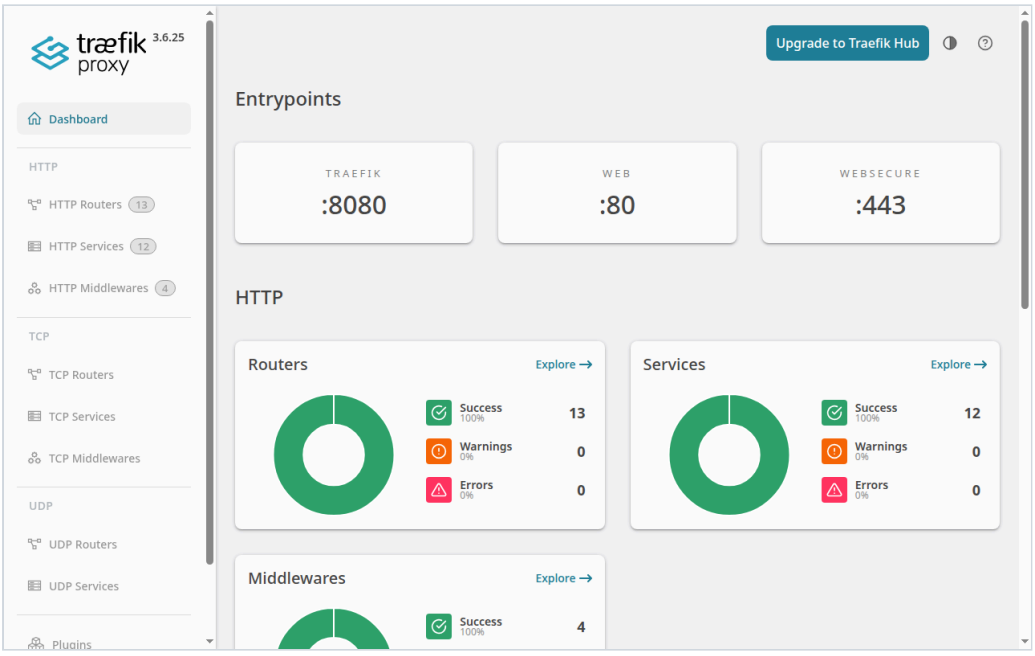
```
$ docker exec swarm-manager-1 docker network create -d overlay --attachable web
$ docker cp traefik.yml swarm-manager-1:/home/manager/traefik.yml
$ docker exec swarm-manager-1 docker stack deploy --detach=true -c /home/manager/traefik.yml traefik
Creating service traefik_traefik
Creating service traefik_whoami

$ docker exec swarm-manager-1 docker stack services traefik
ID             NAME              MODE         REPLICAS   IMAGE                   PORTS
2h5tu673cef8   traefik_traefik   replicated   1/1        traefik:v3.6            *:80->80/tcp
9ucy19b91y73   traefik_whoami    replicated   1/1        traefik/whoami:latest

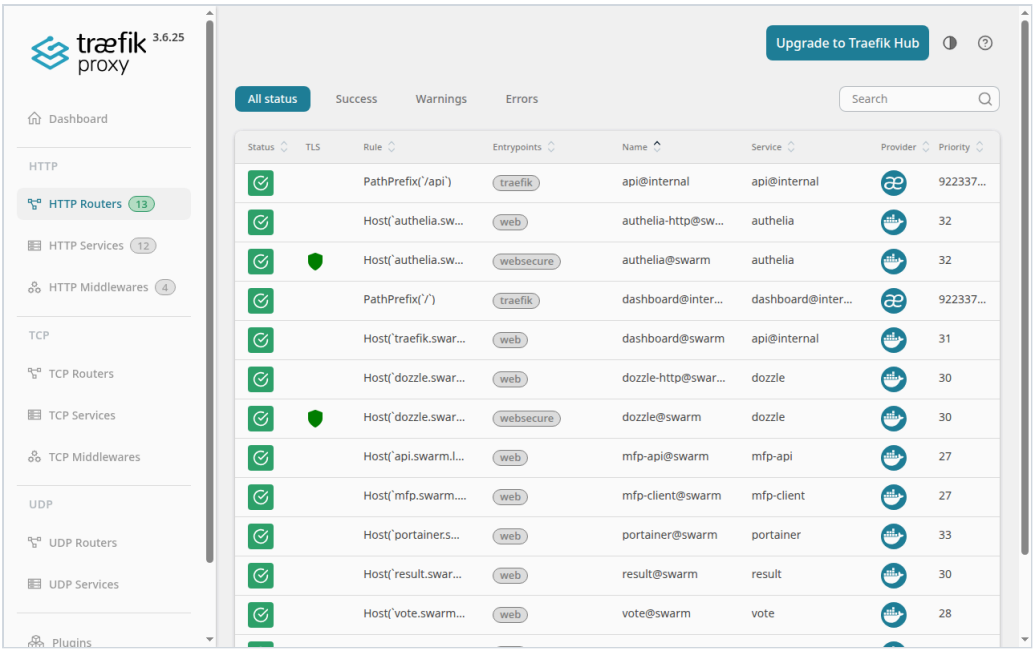
$ curl -s http://whoami.swarm.localhost/
Hostname: de1834f82d3f
...
GET / HTTP/1.1
Host: whoami.swarm.localhost
X-Forwarded-Host: whoami.swarm.localhost
X-Forwarded-Proto: http
```

La requête passe par le fichier hosts (127.0.0.1), puis le port 80 du PC qui est mappé sur le conteneur manager, puis le routing mesh du swarm jusqu'à Traefik, et Traefik choisit le service avec le header `Host`.

```
Hostname: de1834f82d3f
IP: 127.0.0.1
IP: ::1
IP: 10.0.1.6
IP: 172.18.0.3
RemoteAddr: 10.0.1.9:36880
GET / HTTP/1.1
Host: whoami.swarm.localhost
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/156.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/jxl,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9
Cookie: authelia_session=jdE9nxS%OrafXjOZ2YrKX9NED1akf5AR
Sec-Ch-Ua: "Not:A-Brand";v="8", "Chromium";v="156", "Google Chrome";v="156"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Linux"
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Upgrade-Insecure-Requests: 1
X-Forwarded-For: 10.0.0.2
X-Forwarded-Host: whoami.swarm.localhost
X-Forwarded-Port: 80
X-Forwarded-Proto: http
X-Forwarded-Server: c5ac374fa666
X-Real-Ip: 10.0.0.2
```



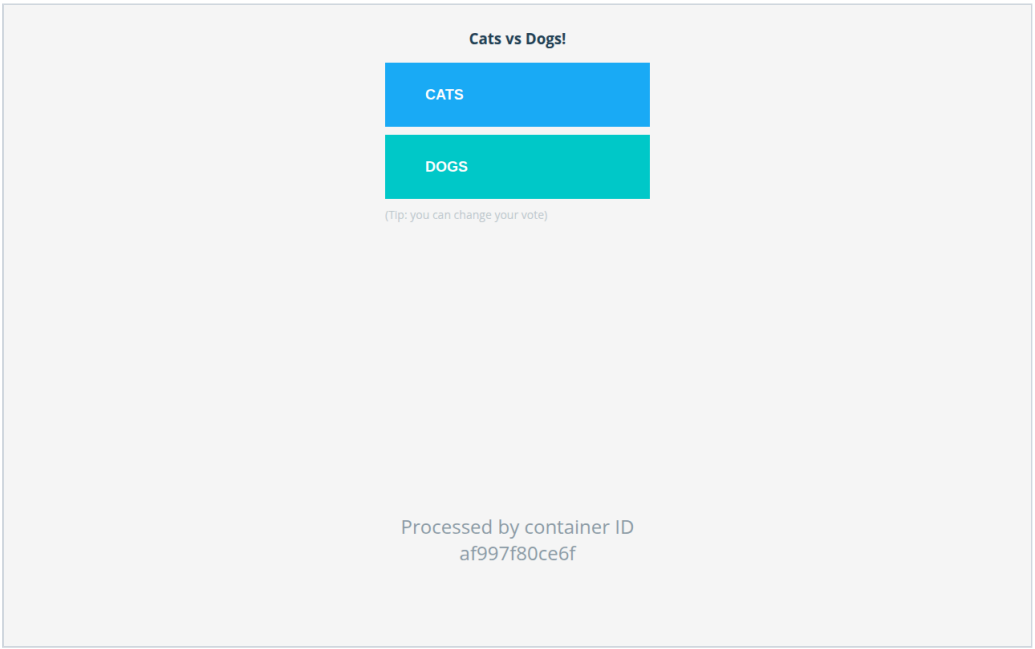
Les routers sont créés à partir des labels des services, il n'y a pas de config centrale.



Exercice 2 Voting app

1. Test en compose sur mon PC, hors swarm

```
$ docker compose ps
SERVICE   STATUS                    PORTS
db        Up 45 seconds (healthy)   5432/tcp
redis     Up 44 seconds (healthy)   6379/tcp
result    Up 39 seconds             127.0.0.1:9229->9229/tcp, 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
vote      Up 39 seconds (healthy)   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
worker    Up 39 seconds
```



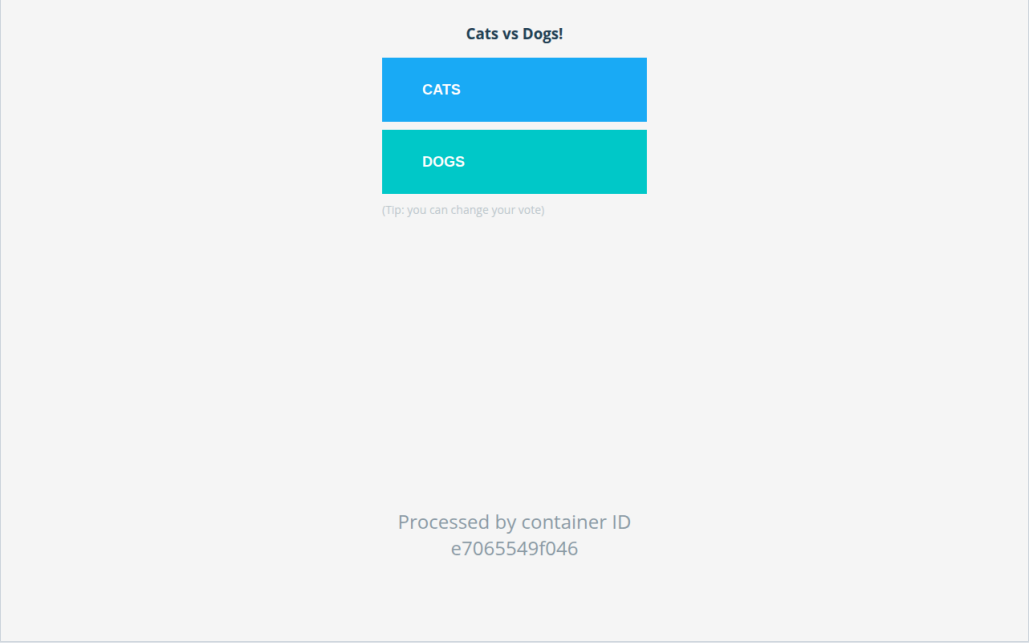
`vote` (python) écrit le vote dans `redis`, `worker` (.NET) le lit et l'enregistre dans `db` (postgres), `result` (node) lit `db` et affiche le résultat. Le réseau `frontend` relie vote, redis et worker, le réseau `backend` relie worker, db et result.

2 et 3. Dans `swarm/voting.yml` j'ai enlevé les ports publiés de vote et result, ajouté le réseau `web` et les labels traefik.

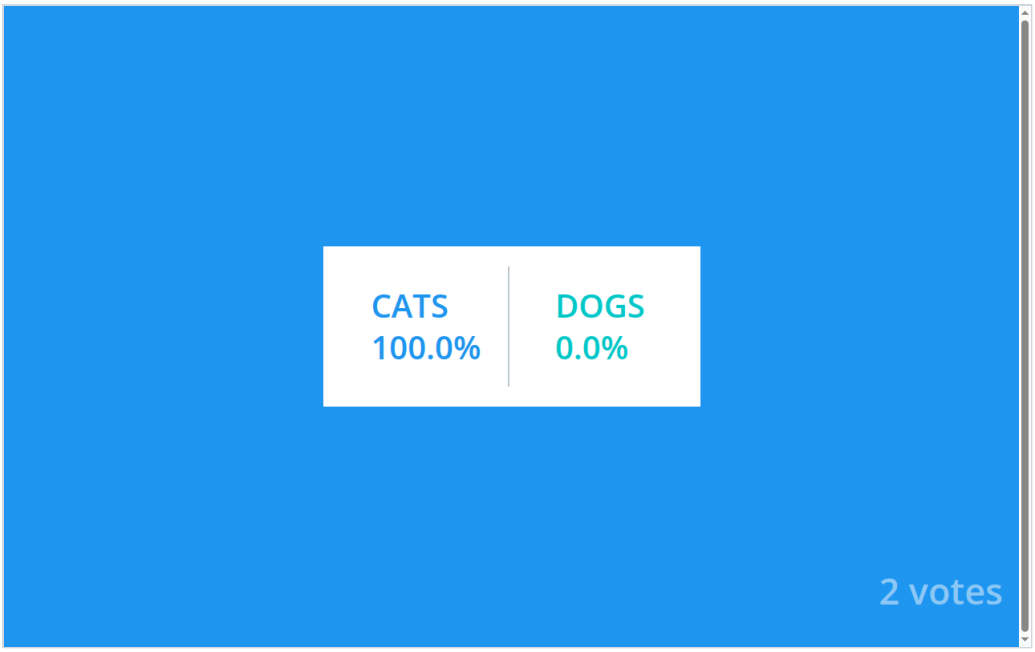
```
vote:
  image: dockersamples/examplevotingapp_vote
  networks:
    - frontend
    - web
  deploy:
    replicas: 2
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.vote.rule=Host(`vote.swarm.localhost`)"
      - "traefik.http.routers.vote.entrypoints=web"
      - "traefik.http.services.vote.loadbalancer.server.port=80"
```

```
$ docker exec swarm-manager-1 docker stack ps vote --filter desired-state=running ...
NAME            NODE           CURRENT STATE            ERROR
vote_db.1       de69559d8a5a   Running 40 seconds ago
vote_redis.1    ddc4456364e2   Running 43 seconds ago
vote_result.1   d0ada8f4a045   Running 45 seconds ago
vote_vote.1     manager        Running 39 seconds ago
vote_vote.2     ddc4456364e2   Running 40 seconds ago
vote_worker.1   d0ada8f4a045   Running 43 seconds ago
vote_worker.2   ddc4456364e2   Running 43 seconds ago
```

Les services sont répartis sur les 4 noeuds.



J'ai voté sur vote.swarm.localhost et le vote apparaît sur result.swarm.localhost.



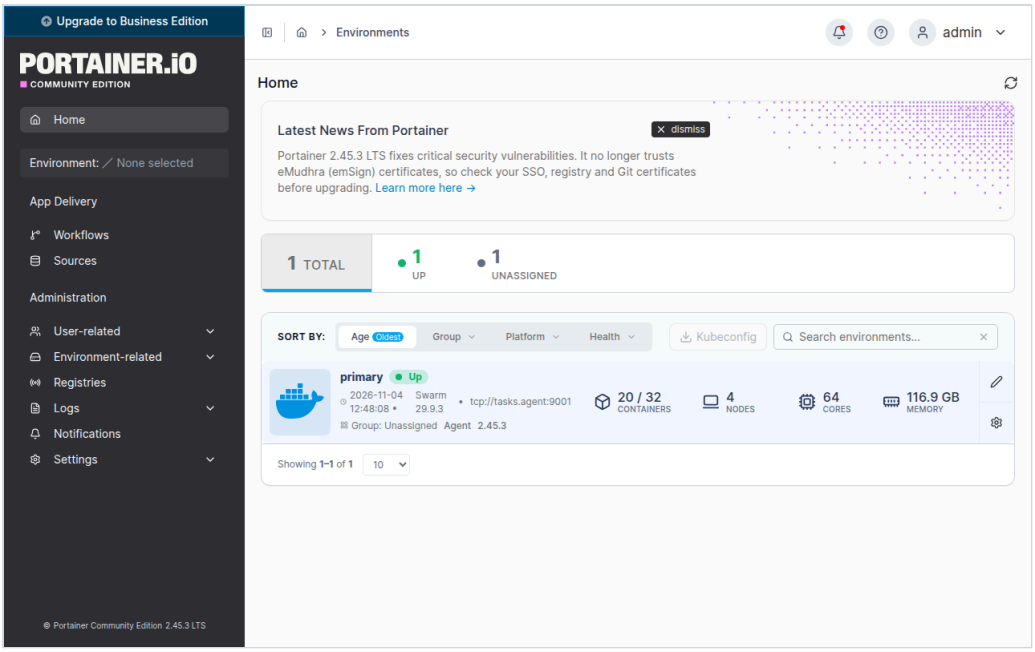
Exercice 3 Portainer

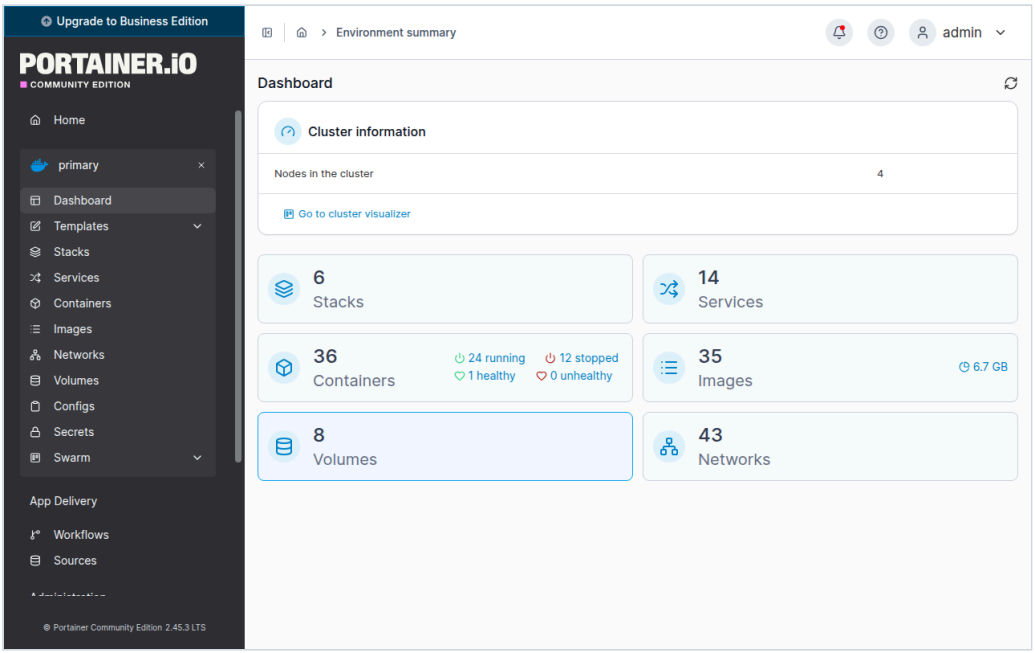
`swarm/portainer.yml` c'est la stack de la doc officielle, j'ai ajouté le réseau `web` et les labels traefik vers le port 9000.

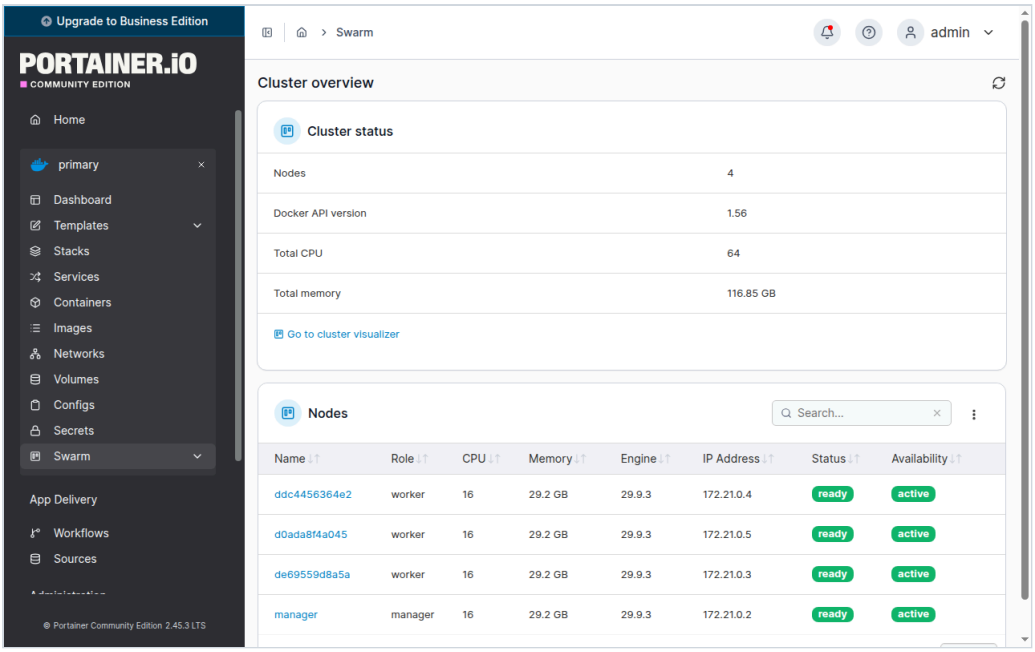
```
$ docker exec swarm-manager-1 docker stack deploy --detach=true -c /home/manager/portainer.yml
portainer
Creating network portainer_agent_network
Creating service portainer_agent
Creating service portainer_portainer

$ docker exec swarm-manager-1 docker stack services portainer
ID             NAME                  MODE         REPLICAS   IMAGE                          PORTS
v87x4dty41fm   portainer_agent       global       4/4        portainer/agent:lts
75tmetgus6zx   portainer_portainer   replicated   1/1        portainer/portainer-ce:lts   *:8000-
>8000/tcp, *:9000->9000/tcp, *:9443->9443/tcp
```

L'agent tourne sur chaque noeud (global) et l'interface sur le manager. A la création du compte admin cette version demande un setup token, il est dans les logs du service.


Upgrade to Business Edition
PORTAINER.IO
COMMUNITY EDITION
Home
Environment: None selected
App Delivery
Workflows
Sources
Administration
User-related
Environment-related
Registries
Logs
Notifications
Settings
© Portainer Community Edition 2.45.3 LTS
Environments
admin
Home
Latest News From Portainer
dismiss
Portainer 2.45.3 LTS fixes critical security vulnerabilities. It no longer trusts eMudhra (emSign) certificates, so check your SSO, registry and Git certificates before upgrading. Learn more here →
1 TOTAL
1 UP
1 UNASSIGNED
SORT BY:
Age Oldest
Group
Platform
Health
Kubeconfig
Search environments...
primary
Up
2026-11-04 12:48:08
Swarm 29.9.3
tcp://tasks.agent:9001
20 / 32 CONTAINERS
4 NODES
64 CORES
116.9 GB MEMORY
Group: Unassigned Agent 2.45.3
Showing 1–1 of 1
10


Upgrade to Business Edition
PORTAINER.IO
COMMUNITY EDITION
Home
primary
Dashboard
Templates
Stacks
Services
Containers
Images
Networks
Volumes
Configs
Secrets
Swarm
App Delivery
Workflows
Sources
© Portainer Community Edition 2.45.3 LTS
Environment summary
admin
Dashboard
Cluster information
Nodes in the cluster
4
Go to cluster visualizer
6 Stacks
14 Services
36 Containers
24 running
12 stopped
1 healthy
0 unhealthy
35 Images
6.7 GB
8 Volumes
43 Networks



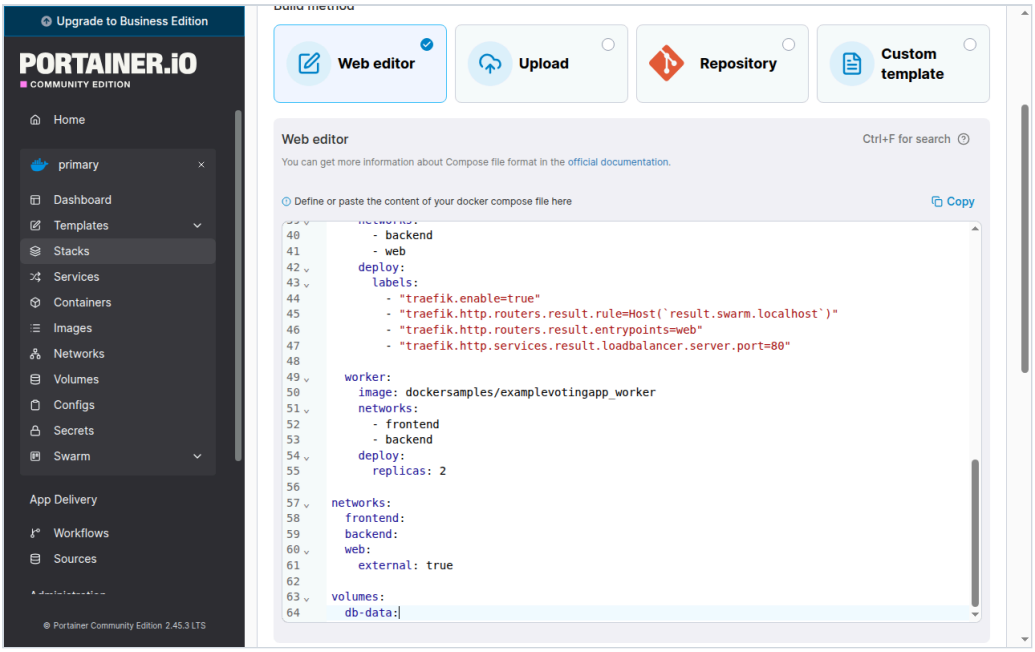
Exercice 4 Redéploiement

1. Suppression de la stack vote en ligne de commande

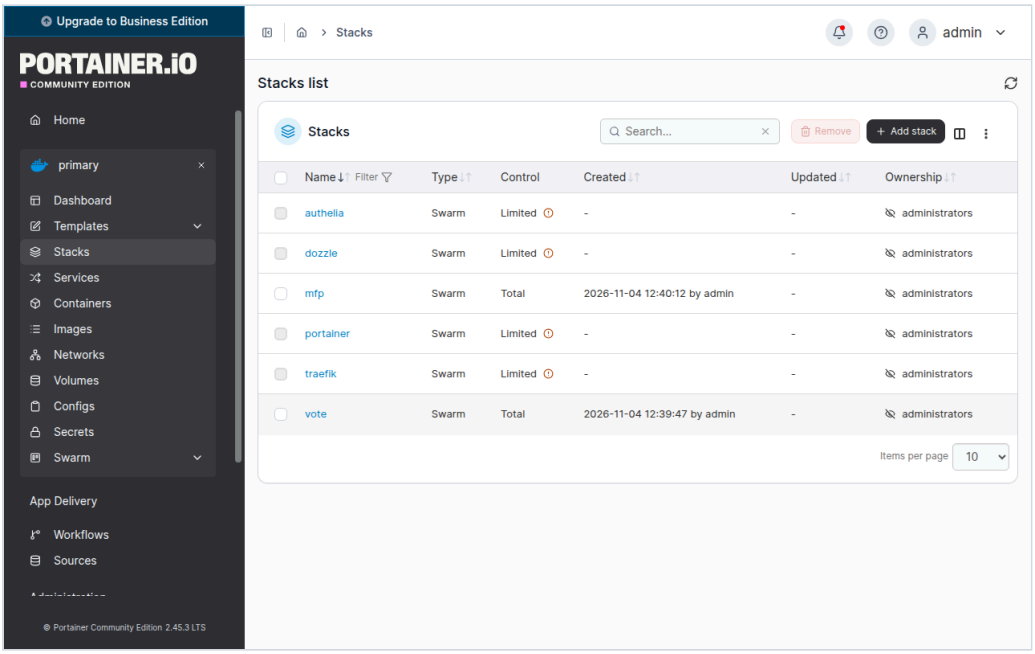
```
$ docker exec swarm-manager-1 docker stack rm vote
Removing service vote_db
Removing service vote_redis
Removing service vote_result
Removing service vote_vote
Removing service vote_worker
Removing network vote_frontend
Removing network vote_backend

$ docker exec swarm-manager-1 docker stack ls
NAME        SERVICES
portainer   2
traefik     2
```

2 et 3. Dans Portainer, Stacks > Add stack > Web editor, j'ai collé `voting.yml`.



vote.swarm.localhost répond à nouveau. Les stacks créées depuis Portainer (vote et mfp) sont en contrôle Total, celles déployées en ligne de commande sont en Limited.



4. `swarm/mfp.yml` met l'API sur api.swarm.localhost, et j'ai aussi mis le front sur mfp.swarm.localhost. Stack déployée depuis Portainer.

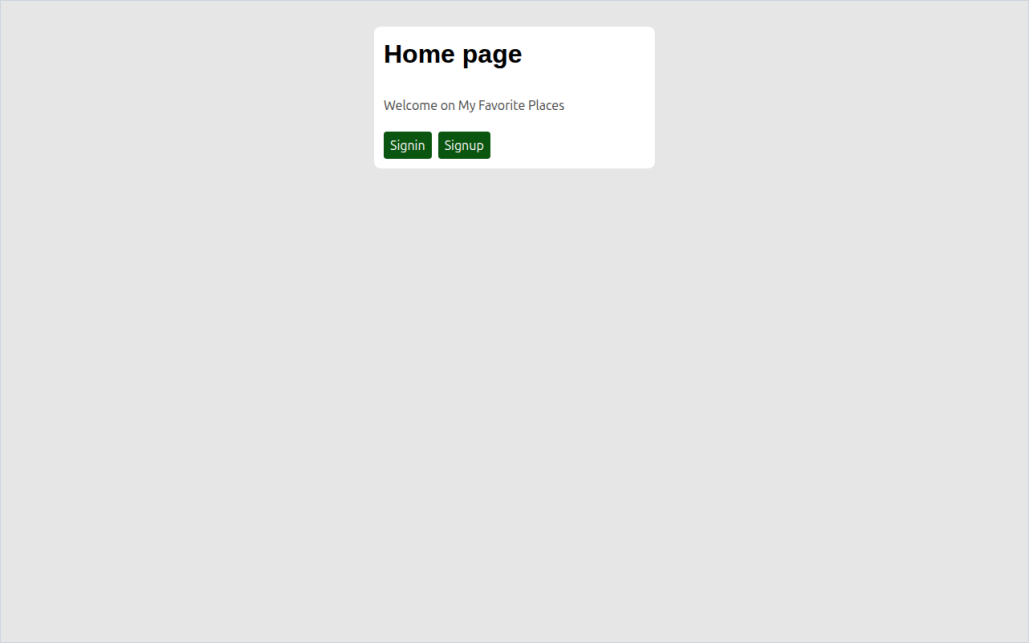
```
$ docker exec swarm-manager-1 docker stack ps mfp ...
NAME               NODE           DESIRED STATE   CURRENT STATE            ERROR
mfp_client.1       de69559d8a5a   Running         Running 34 seconds ago
 \_ mfp_client.1   de69559d8a5a   Shutdown        Failed 40 seconds ago    "task: non-zero exit (1)"
mfp_client.2       d0ada8f4a045   Running         Running 39 seconds ago
mfp_db.1           manager        Running         Running 37 seconds ago
mfp_server.1       d0ada8f4a045   Running         Running 33 seconds ago
 \_ mfp_server.1   d0ada8f4a045   Shutdown        Failed 38 seconds ago    "task: non-zero exit (1)"
mfp_server.2       ddc4456364e2   Running         Running 35 seconds ago
 \_ mfp_server.2   ddc4456364e2   Shutdown        Failed 40 seconds ago    "task: non-zero exit (1)"
```

Il y a des tâches en échec au début. Swarm ignore `depends_on`, donc le serveur démarre avant la BDD, s'arrête avec le code 1 et Swarm le relance jusqu'à ce que la BDD réponde (c'est pour ça que j'avais corrigé le code de sortie). Pareil pour le client, nginx refuse de démarrer tant que `server` n'existe pas.

Test de l'API avec curl en passant par Traefik

```
$ curl -X POST http://api.swarm.localhost/api/users -H "Content-Type: application/json" -d
'{"email":"pelavo@swarm.localhost","password":"demo-password"}'
{"item":{"id":1,"email":"pelavo@swarm.localhost","hashedPassword":"<hash>","createdAt":"2026-11-
04T11:41:00.061Z"}}
HTTP 200

POST /api/users/tokens HTTP 200
GET /api/users/me sans token HTTP 403
client HTTP 200
client vers API via nginx HTTP 403
```



Cours 5 Secrets et configs

HTTPS

Authelia exige un `authelia_url` en https, et le flow OIDC redirige vers des URL en https. Le sujet donne des URL en http mais ça ne marche pas sans TLS, donc j'ai ajouté un entrypoint `websecure` sur le 443 dans Traefik avec un certificat auto-signé pour `*.swarm.localhost`. Le http redirige vers le https pour dozzle et authelia. Le navigateur affiche un avertissement à cause du certificat.

Le certificat est mis dans une config et la clé privée dans un secret.

J'ai dû recréer le conteneur manager pour ouvrir le 443, le cluster a gardé son état (même IP, même `docker node ls`).

Secrets et configs

Authelia lit ses secrets dans des fichiers (variables `*_FILE`) et le filtre `template` met le reste dans la config, donc `swarm/authelia-config.yml` ne contient aucune valeur sensible.

```
$ docker exec -i swarm-manager-1 docker secret create authelia_session_secret - <
.local/authelia_session_secret
lgtukr2lq2l2ygdoc89usl7b9
$ docker exec -i swarm-manager-1 docker config create traefik_tls - < traefik-tls.yml
i2o9tdxtcfj6scaui2gfjahy5
```

On ne peut pas lire un secret, alors qu'on peut lire une config

```
$ docker exec swarm-manager-1 docker secret inspect authelia_session_secret --format "{{json .Spec}}"
{"Name":"authelia_session_secret","Labels":{}}

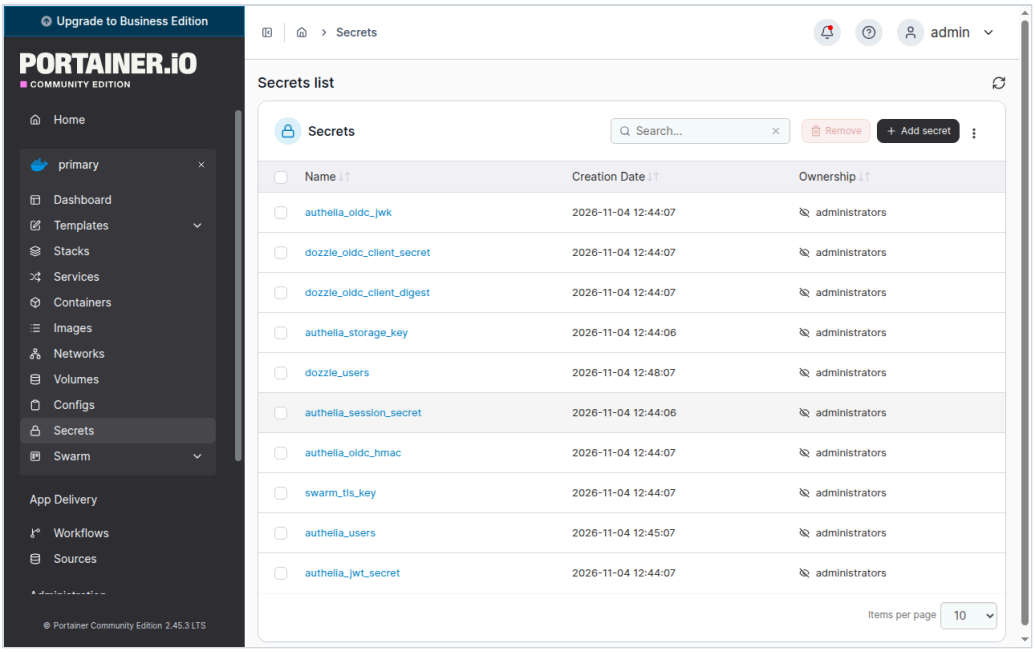
$ docker exec swarm-manager-1 docker config inspect --pretty traefik_tls
Name:                   traefik_tls
Data:
tls:
  stores:
    default:
      defaultCertificate:
        certFile: /etc/traefik/swarm.crt
        keyFile: /run/secrets/swarm_tls_key
```

On ne peut pas non plus modifier un secret. Je me suis trompé dans le fichier users d'Authelia et j'ai dû faire

```
$ docker exec swarm-manager-1 docker stack rm authelia
$ docker exec swarm-manager-1 docker secret rm authelia_users
$ docker exec -i swarm-manager-1 docker secret create authelia_users - < .local/authelia_users.yml
$ docker exec swarm-manager-1 docker stack deploy --detach=true -c /home/manager/authelia.yml authelia
```

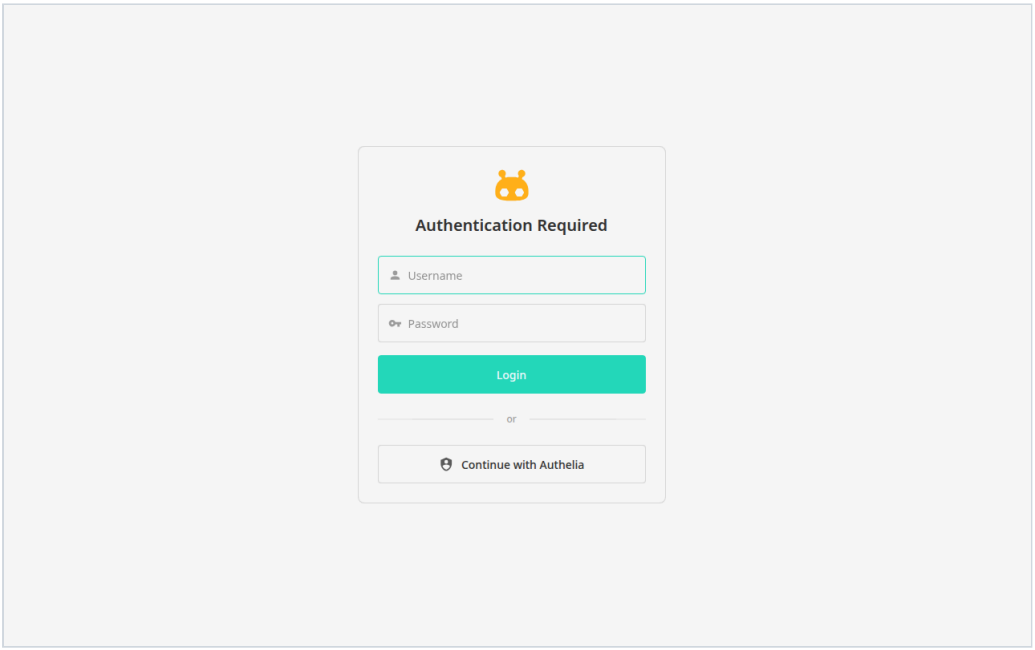
Exercice 1.1 Dozzle

J'ai généré `users.yml` avec `dozzle generate`, puis créé le secret `dozzle_users` dans Portainer (Secrets > Add secret). Il est monté sur `/data/users.yml` dans `swarm/dozzle.yml`.

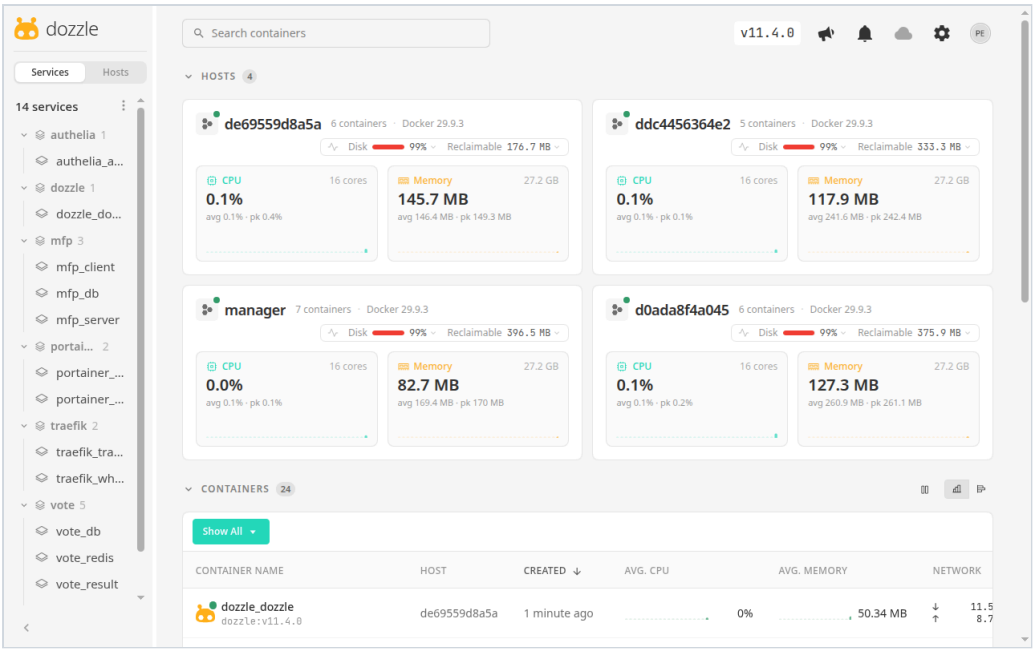



```
$ docker exec swarm-manager-1 docker stack services dozzle
ID             NAME             MODE      REPLICAS   IMAGE                    PORTS
hjl4q32x85oa   dozzle_dozzle    global    4/4        amir20/dozzle:v11.4.0
```

Dozzle tourne en global (`DOZZLE_MODE=swarm`) pour avoir les logs de tous les noeuds. Chaque instance a son propre secret de session, donc j'ai mis un cookie sticky dans Traefik pour que le navigateur reste sur la même instance.



Connecté avec le mot de passe de `users.yml`



Exercice 1.2 Authelia et OIDC

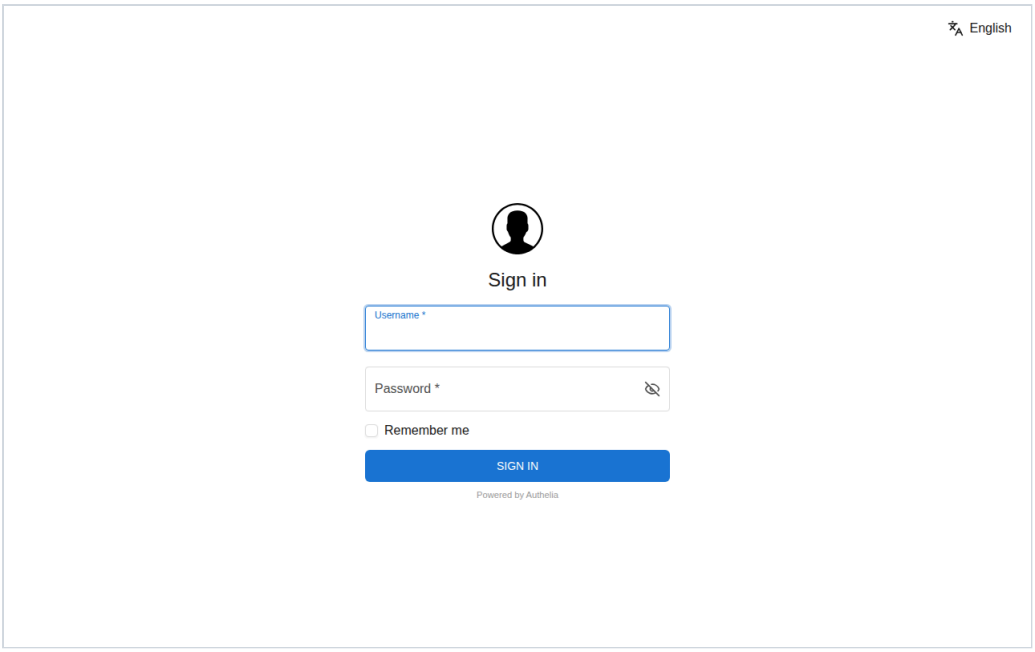
`swarm/authelia.yml` déploie Authelia avec le secret `authelia_users` pour le compte et la config `authelia_config` qui déclare le client OIDC `dozzle`

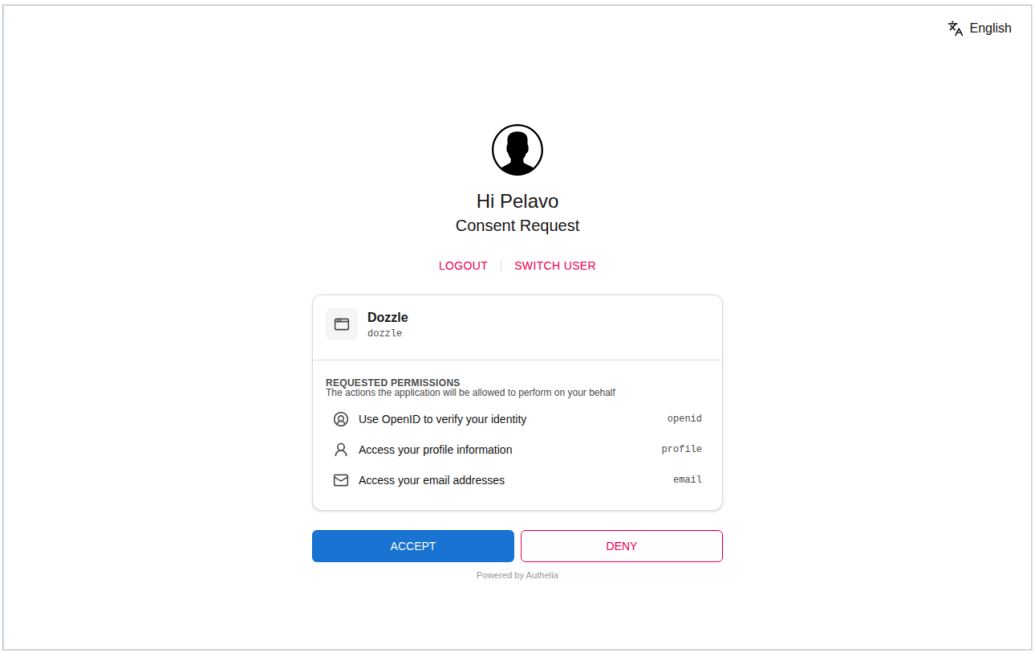
```
clients:
  - client_id: 'dozzle'
    client_name: 'Dozzle'
    client_secret: '{{ secret "/run/secrets/dozzle_oidc_client_digest" }}'
    authorization_policy: 'one_factor'
    redirect_uris:
      - 'https://dozzle.swarm.localhost/api/auth/callback'
    scopes: ['openid', 'profile', 'email']
```

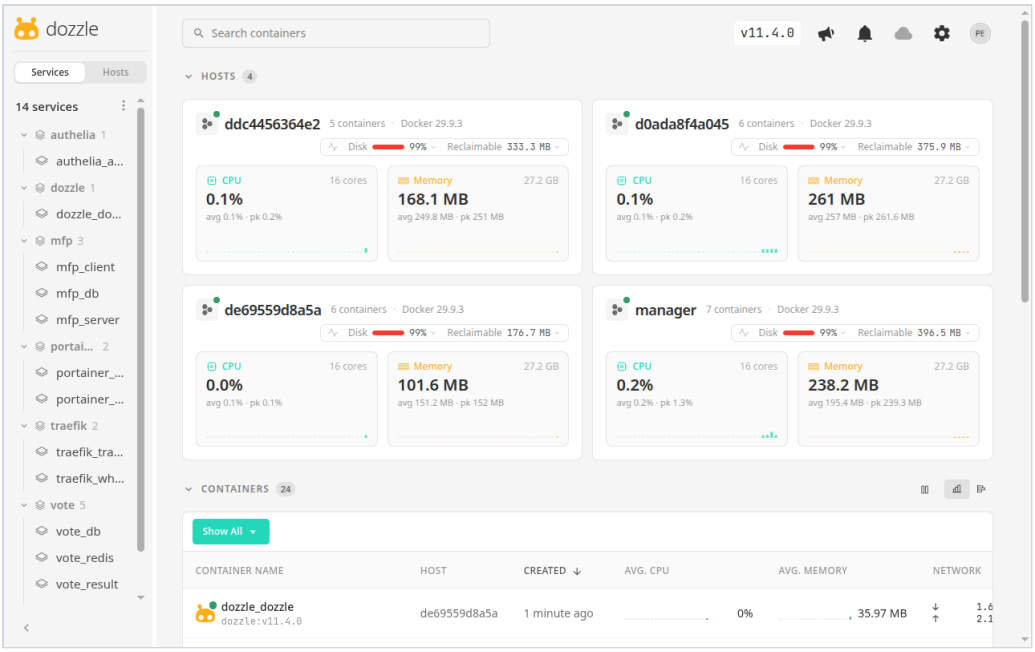
Dozzle reste en provider `simple`, `users.yml` garde la liste des comptes autorisés et le lien avec Authelia se fait par l'email. Pour que Dozzle puisse joindre Authelia depuis le cluster, Traefik a l'alias réseau `authelia.swarm.localhost`, et `SSL_CERT_FILE` fait accepter le certificat à Dozzle.

```
$ curl -sk https://authelia.swarm.localhost/.well-known/openid-configuration | jq "{issuer,
authorization_endpoint, token_endpoint}"
{
  "issuer": "https://authelia.swarm.localhost",
  "authorization_endpoint": "https://authelia.swarm.localhost/api/oidc/authorization",
  "token_endpoint": "https://authelia.swarm.localhost/api/oidc/token"
}
```

Test dans le navigateur, clic sur "Continue with Authelia" dans Dozzle, connexion sur Authelia, consentement, puis retour sur Dozzle connecté avec les 4 noeuds et les 14 services.


English
Sign in
Username *
Password *
Remember me
SIGN IN
Powered by Authelia


English
Hi Pelavo
Consent Request
LOGOUT
SWITCH USER
Dozzle
dozzle
REQUESTED PERMISSIONS
The actions the application will be allowed to perform on your behalf
Use OpenID to verify your identity
openid
Access your profile information
profile
Access your email addresses
email
ACCEPT
DENY
Powered by Authelia



Dans les logs de Dozzle on voit `"provider":"oidc"`

```
{"level":"info","version":"v11.4.0","user":"pelavo","provider":"oidc","time":"2026-11-04T11:49:16Z","message":"Token created"}
```